

FlowGuard 技术架构与制作说明

2026 创青春 AI 黑客松 · 顺德行 · 自由创新赛道 · 一人团队独立完成

FlowGuard 是一款面向中小外贸出纳的跨境付款「先核查、再付款」风控控制台。它在把付款指令提交给持牌机构之前，先做退回风险预检、通道比价选路、双人审批与里程碑条件结算。本文说明其技术架构、核心工作流程与制作过程。

合规底线：FlowGuard 是纯软件决策支持工具——不持牌、不经手资金、不做加密兑换/转账，所有资金结算均由持牌金融机构完成。本文所述「生成付款指令」均指生成可交由持牌机构执行的指令文件，平台任何环节都不流经资金。

一、系统概述

FlowGuard 采用 Next.js App Router 的全栈一体架构：同一个应用内同时承载移动端优先的交互界面（客户端组件）与服务端 API 路由（Route Handlers）。风控的确定性内核在服务端与共享库中运行，AI 作为补充信号层通过 App AI 计费代理调用，用户身份、数据库与对象存储均由 Eazo 平台能力提供。

设计原则

- 先拦截，而非先失败：在提交指令前给出退回概率与卡点，而不是在失败后对账。
- 规则是骨架，AI 是补充：确定性规则引擎给出可解释、可复现的评分；AI 只加警示、绝不降低评分。
- 职责分离（Maker-Checker）：出纳提交、复核人第二签，服务端硬性禁止自我批准；双人放行为下限，大额付款可按额度升级为多级/多签审批。
- 移动端优先：所有界面按手机视口设计，几步走完一笔高风险付款的完整控制。
- 双语内建：全部用户可见文案经 react-i18next 的 t() 渲染，en-US / zh-CN 结构对齐。

二、技术架构

技术栈总览

层次	技术选型	说明
框架	Next.js 16 (App Router) + React 19	服务端渲染 + 客户端交互一体化
语言	TypeScript 5 (strict)	全站类型安全，tsc 零错误
样式	Tailwind CSS 4 + tw-animate-css	移动端优先、设计令牌统一
动效	framer-motion	步骤切换与卡片过渡
组件	shadcn / @base-ui/react + lucide-react	无样式基元 + 图标
国际化	i18next / react-i18next	en-US / zh-CN 双语键对齐
校验	zod	API 入参与快照结构校验

层次	技术选型	说明
数据库	Eazo 托管 PostgreSQL + drizzle-orm + postgres	服务端 DATABASE_URL，类型化查询
AI	Eazo App AI 计费代理（DeepSeek）	通过 /api/ai 接口按能力路由
平台能力	@eazo/sdk	内建鉴权、App AI、对象存储
可被 AI 调用	@modelcontextprotocol/sdk	暴露 MCP 服务端，供外部 Agent 调用
通知	qrcode + 邮件摘要接口	收据/核实二维码、每日摘要

分层架构

FlowGuard 从上到下分为四层，数据单向下沉、结果单向上浮：

层	职责	关键位置
表现层	移动端界面、向导步骤、路演/文档页	src/app/**、src/components/screens、src/components/wizard
应用层	客户端 API 封装、状态与数据提供者、里程碑 store	src/lib/api、src/components/shell/data-provider、src/lib/milestones
服务层	Route Handlers：付款/供应商/审批/AI/公开收据/MCP	src/app/api/**
领域层	确定性风控引擎、失败案例库、供应商输入建模、分析	src/lib/engine、src/lib/verification、src/lib/analytics

目录结构（关键）

- src/app/ — 页面路由（首页、pay、review、milestones、cases、history、reconcile、suppliers、公开 receipt/case、hackathon 路演）与 api/ 服务端路由。
- src/components/ — screens/（各页面主体）、wizard/（付款向导：build/preccheck/route 步骤）、ai/（AI 信号与简报卡）、shell/（应用外壳、导航、数据提供者）、hackathon/（路演内容与幻灯片）。
- src/lib/ — engine/（风控内核）、api/（前端请求封装）、db/（drizzle schema、queries、seed）、milestones/（里程碑 store）、verification.ts、analytics.ts、i18n/。
- scripts/ — 文档 PDF 生成器、PPTX 生成器、真实页面截图脚本（Playwright）。
- public/ — 生成的交付物（PDF、PPTX、截图）。

三、数据模型

核心实体及其关系：

实体	关键字段	说明
Supplier（收款人）	国家/币种、SWIFT/IBAN、账户状态、受限地区、历史退回率	风控评分的主要输入

实体	关键字段	说明
Payment（付款指令）	供应商、金额、结算币种、通道、状态、Maker/Checker	从草稿到到账的全生命周期载体
RiskAssessment（风险评估）	评分、退回概率、命中因子、退回成本	规则引擎的确定性输出
RoutingResult（选路结果）	多类持牌结算通道的费用/时效/退回率排序	可解释的最优路径推荐
VerificationCase（核实工单）	供应商、命中因子、状态、公开 token	数据质量类风险的闭环
Milestone/Program（里程碑）	项目、条件、金额、状态流转、证据	条件结算工作台
Receipt（到账确认）	公开 token、二维码	供收款方查看的到账确认页

数据库通过 drizzle-orm 定义 schema（src/lib/db/schema），以类型化查询访问（src/lib/db/queries），DATABASE_URL 仅存于服务端。访客模式（Guest）下，用户自己发起的付款仅存于本设备 localStorage，离开访客模式即清除。

四、核心功能与工作流程

FlowGuard 由四大核心能力构成，从「先核查」到「透明追踪」形成闭环：

4.1 收款方信息预检-AI Agent（功能①）

面向收款账户信息的智能校验、国别规则适配、退回风险评分与合规风险提示报告。

1. 出纳在 /pay 选择收款人、填写金额与结算币种，提交草稿。
2. 前端调用 POST /api/payments/assess，服务端调用规则引擎产出 RiskAssessment 与 RoutingResult。
3. 账户信息智能校验：收款名 / SWIFT / IBAN 一致性与账户状态校验；先拦截而非先失败。
4. 国别规则适配：按收款人国家 / 币种自动适配该走廊的结算要求（收款名匹配、SWIFT/IBAN、FX 管制文件等）。
5. 退回风险评分：确定性规则引擎给出退回概率与命中因子；AI（DeepSeek，经 App AI 代理 POST /api/ai/insight）按需补充语义 / 情境风险信号——只加警示、绝不降低评分，不可用时确定性报告不受影响。
6. 合规风险提示报告：把检查项翻译成通俗解释与修复步骤，数据质量类因子可一键生成核实工单（VerificationCase），核实后自动清除 / 软化。

4.2 多路径智能路由推荐引擎（功能②）

统一池管理多类持牌结算通道，仅输出最优路径对比建议，不执行付款。

1. 预检通过后进入 /pay 的选路步骤，展示 RoutingResult。
2. 统一通道池：传统电汇、跨境人民币、第三方跨境支付通道；境外主体的可选通道列表包含境外持牌数字结算服务商方案。
3. 多维度比价：按费用、时效、退回率对各持牌通道自动排序，给出可解释的最优路径推荐。
4. 只建议、不执行：平台仅输出最优路径对比建议，实际付款由银行 / 持牌机构执行——平台不经手资金。

4.3 分阶段里程碑付款工作台（条件结算管理，功能③）

外包 / 项目付款节点台账管理，按里程碑进度设置付款触发条件。

1. 在 /milestones 按项目管理付款里程碑及其触发条件（外包 / 项目场景）。
2. 节点台账管理：付款节点集中管理，进度与金额一目了然。
3. 里程碑触发条件：状态流转 待开始 → 进行中 → 待校验 → 已校验；里程碑状态存于客户端 store（useSyncExternalStore + localStorage，已做快照缓存避免无限渲染）。
4. 完成校验 → 放款提醒：校验通过后向财务推送放款提醒，深链跳转 /pay 并预填供应商与金额，仍走完整预检与选路。
5. 合规边界：仅做条件与提醒管理，实际放款操作全部在银行 / 持牌结算机构完成——平台不托管、不放款。

4.4 统一结算状态与对账看板（功能④）

聚合各通道汇款进度、中转链路信息、交易凭证，实现多笔跨国付款台账可视化与自动对账核销。

1. 在 /reconcile 聚合各持牌通道的汇款进度与中转链路信息，钱走到哪一目了然。
2. 凭证与台账可视化：聚合交易凭证（dispatch 记录提交、proof 上传凭证、到账后生成公开收据 Receipt），多笔跨国付款台账集中可视化。
3. 自动对账核销：应收 vs 实收、费用与汇兑损失自动匹配核销，并可导出对账单（CSV）。
4. 结算链路透明：结算全程可追踪；平台只做透明化追踪，不经手资金。

说明：Maker-Checker 双人审批作为底层内控贯穿①—③（付款指令在 /review 需第二签放行、服务端硬性禁止自我批准），是四大功能之下的通用控制点。双人放行为下限；大额付款可按额度阈值升级为多级/多签审批（即使在企业内部）。

五、API 接口清单

接口	方法	用途
/api/payments/assess	POST	运行退回风险预检与选路
/api/payments	GET/POST	付款指令列表 / 创建
/api/payments/[id]/review	POST	第二签审批（禁止自我批准）
/api/payments/[id]/dispatch	POST	记录已提交至持牌机构
/api/payments/[id]/proof	POST	上传付款凭证
/api/ai/insight	POST	AI 补充风险信号（App AI 代理）
/api/precheck/explain	POST	AI 合规简报解释
/api/suppliers、/api/suppliers/[id]	GET/...	收款人台账
/api/cases、/api/verification-cases/[id]	GET/...	核实工单
/api/public/receipt/[token]	GET	公开到账确认页数据

接口	方法	用途
/api/public/case/[token]	GET	公开核实页数据
/api/erp/payments	GET	只读 ERP 对账数据
/api/mcp	POST	MCP 服务端，供外部 AI Agent 调用
/api/notifications/...	POST	每日摘要 / 测试通知

所有涉及用户数据的接口默认要求鉴权，并强制按用户隔离（用户 A 无法读写用户 B 的数据）。

六、AI 与合规边界

- AI 定位：四个真实痛点各绑定一个 AI 能力——看不到钱卡哪（资金流解读）、退回事后才知道（起草核实消息）、多国重复适应银行（走廊要求清单）、对账对不上（差异归因）。
- 调用方式：面向使用者的 App AI 通过 Eazo 计费代理路由（EAZO_APP_AI 相关环境变量），按能力选择模型，密钥不落客户端。
- 合规四不碰：不持牌、不经手资金、不做加密兑换/转账、由持牌机构结算。全站措辞已统一为「提交指令 / 到账确认」，并附双语《合规审查报告》PDF。

七、制作说明

7.1 本地运行

- 包管理与运行：bun（bun run dev 启动开发服务器，Next.js 热更新）。
- 类型检查：tsc --noEmit（零错误）；代码规范：eslint（零告警）。
- 数据库：drizzle-kit 生成/推送 schema，DATABASE_URL 由 Eazo 托管 PostgreSQL 提供。

7.2 交付物生成脚本

交付物	生成脚本	说明
本技术文档 PDF	scripts/gen-compliance-pdf.js	通用 Markdown→PDF 渲染器，支持标题/列表/引用/表格
合规审查报告 / 用户指南 PDF	scripts/gen-compliance-pdf.js	同一渲染器，传入不同 Markdown 与页脚
黑客松路演 PPTX	scripts/gen-hackathon-pptx.py	python-pptx 构建 13 页；功能页左文右图内嵌真实截图
功能页面真实截图	scripts/shoot-feature-pages.js	Playwright 无头 Chromium 驱动真实页面截图

7.3 PDF 渲染要点

- 使用 pdfkit，注册 Noto Sans CJK 字体以支持中文；A4、预留页脚带。

- 关闭对自动分页的依赖：以 `ensure()` 手动判断剩余高度并主动 `addPage()`，页脚阶段将底边距置 0，避免此前出现的空白页/溢出。
- 字体不含 emoji 字形，`inline()` 会将 emoji 与变体选择符转换为安全符号（✓、×、△），避免豆腐块。

7.4 真实截图流程

- 以移动端视口（402×1180、2x）启动无头浏览器，先通过访客门、切换中文界面。
- 驱动 `/pay` 向导（高风险供应商）到预检结果，截取预检与 AI 信号卡。
- 端到端提交一笔低风险付款作为种子数据，使 `/review` 出现待批指令后再整页截图。
- `/milestones` 直接整页截图（其状态 `store` 已做快照缓存，避免无限渲染）。

7.5 工程质量

- 全站 TypeScript strict，`tsc --noEmit` 零错误；ESLint 零告警。
- 双语键 `en-US / zh-CN` 结构 100% 对齐，用户可见文案不硬编码、统一经 `t()`。
- 关键路由（`pay / review / milestones / cases / history / 公开收据`）端到端可跑通。

八、关键文件索引

主题	位置
风控规则引擎	<code>src/lib/engine/index.ts</code> 、 <code>src/lib/engine/failure-cases.ts</code> 、 <code>types.ts</code>
付款向导	<code>src/components/wizard/build-step.tsx</code> 、 <code>precheck-step.tsx</code> 、 <code>route-step.tsx</code>
AI 信号与简报	<code>src/components/ai/ai-risk-signals.tsx</code> 、 <code>ai-insight-card.tsx</code>
审批与历史	<code>src/components/screens/review-screen.tsx</code> 、 <code>history</code> 相关屏
里程碑工作台	<code>src/lib/milestones/milestone-store.ts</code> 、 <code>src/components/screens/milestones-screen.tsx</code>
数据库	<code>src/lib/db/schema</code> 、 <code>src/lib/db/queries</code> 、 <code>src/lib/db/seed-suppliers.ts</code>
国际化	<code>src/i18n/locales/en.json</code> 、 <code>zh.json</code>
路演与文档	<code>src/components/hackathon/*</code> 、 <code>scripts/gen-hackathon-pptx.py</code> 、 <code>scripts/gen-compliance-pdf.js</code>